

دانشگاه صنعتی شریف

دانشکده مهندسی برق

درس طراحی سیستم های نهفته بی‌درنگ (25778)

دکتر غلامپور

تمرین سری سوم

عرشیا نوروزی 402170562

نیم سال دوم 05-04

بخش تئوری

سوال 1: معماری کلی یک سیستم ***Embedded Linux***

نقش هر یک از اجزای زیز در یک سیستم *Embedded Linux*، و فرآیند راه اندازی:

معماری یک امبدد لینوکس سیستم، از لایه های مجزا تشکیل شده حه به صورت مرتبه به مرتیبه و واسبته به هم عمل میکنند. در بوت، هر لایه وظیفه آماده سازی بستر سخت افزاری یا نرم افزاری خاصی را بر عهده دارد تا محیط برای لایه ی بعدی مهیا باشد. هر کدام از لایه ها به شرح زیر هستند:

. ***BootLoader***:

اولین کدی است که پس از روشن شدن پردازنده اجرا میشود. وظیفه اصلی اش پیگربندی اولیه سخت افزار های اصلی هست. مانند تنظیم کلاک ردازنده یا کنترلر حافظه ***ram***.

بوت لودر بعد از آماده سازی ***ram***، ***image*** کرنل لینوکس و گاهی ساختار سخت افزار***(device tree)*** را از حافظه (فلش نان والوتایل سیستم) برمیدارد و در ***ram*** کپی میکند. در نهایت با انتقال کنترل پردازنده به آدرس شروع کرنل، زمینه ی اجرای کرنل لینوکس را فراهم می آورد.

. ***Linux Kernel***:

کرنل لینوکس، هسته ی سیستم عاملمان هت و وظیفه مدیریت منابع سخت افزاری، اسکجولینگ پردازش ها، مدیریت حافظه و بارگذاری درایورهای شبکه و سایر پریفرال ها را بر عهده دارد.

هسته به تنهایی قادر به اجرای برنامه ی کاربر نیت و بعد از پیکربندی سخت افزار، ***rootfs*** را پیدا میکند و آن را ***Mount*** میکند تا به فایل های اجرایی و کتابخانه های یوزر اسپیس دسترسی پیدا کند.

. ***Root File System*:**

دایرکتوری استاندارد لینوکس است و شامل */lib, /bin, /etc* میشود که تمامی کتابخانه های و اسکریپت های پیکربندی و برنامه های کاربردی را شامال است. با مونت شدن این بخش، کرنل مسیری برای یافتن برنامه ها دارد و دنبال آن ها میگردد، اکنونو اجرای این برنامه ها نیازمنده دسترسی به کتابخانه های پایه ای است که شامل *C library* میشود.

. *C Library*:

کتابخانه *C*، رابط اصلی برنامه های یوزر اسپیس با کرنل هستند، توابع استانداردی را شامل میشوند که برای *system call* ها، تخصیص حافظه، دسترسی به فایل ها و ایجاد سوکت ها را فراهم میکند. برای استارت کردن *init* یا *system* کرنل نیاز به این لایبرری ها دارد.

. *init/Systemd*:

اولین برنامه یوزراسپیس هست که وظیفه مدیریت سرویس ها، راه اندازی شبکه، *mount* کردن سایر فایل سیستم ها و ایجاد محیط آماده برای کاربر را بر عهده دارد. این برنامه شامل اسکریپت های زیادی است تا با ران کردن آنها سرویس های پس زمینه*(Daemons)* را اجرا کند و محیط سیستم عامل به پایداری کامل برسد و سیستم کاملا بالا بیاید و بستر برنامه های یوزر را فراهم بسازد.

. *User Applications*:

همان برنامه هایی را شامل میشوند که هدف ساخت این سیستم بودند، مثلا کنترل کننده یک موتور یا یک وب سرور خانگی

سوال2: محیط توسعه و *Toolchain*

فرض کنید میخواهید یک برنامه *C++* را برای یک برد *ARM* توسعه دهید، درحالی که روی یک لپتاپ *x86* کار میکنید:

الف) چرا نمیتوانید برنامه را مستقیما روی لپتاپ کامپایل و باینتری حاصل را روی برد اجرا کنید؟ دقیقا چه چیزی اشتباه پیش میورد؟ بر همین اساس تفاوت *Native Compilation* و *Cross Compilation* و مفهوم *Host* و *Target* را توضیح بدهید.

وقتی برنامه را مستقیما روی کامپیوترمان کامپایل بکنیم و بر روی برد اجرا کنیم، بدلیل تفاوت های معماری ها و تفاوت *ISA* این دو معماری، باینری حاصل برای *ARM* کاملا غیرقابل فهم و بی معنی خواهد بود و سیستم عامل روی برد خطا ای در مورد خطای *instruction* میدهد. در این حالت ***Host*** کامپیوتر *x86* ما است که روی آن کد مینویسم و کامپایل میکنیم و ***Target*** برد *ARM*مان هست که فایل اجرایی نهایی را روی آن اجرا میکنیم. در این حالت ما باید *Cross Compilation* انجام دهیم، چون ***cross compilation*** زمانی است که معماری *Host* و *Target* متفاوت است و باید بر روی *Host* جوری کامپایل کنیم که باینری ای مطابق با *ISA* *ARM* تولید کند.

Native Compilation زمان ای است که *Host*و *Target* معماری یکسانی دارند، برای مثال لپتاپ *x86* ما برای خودش یا کامپیوتر *x86* دیگری کامپایل میکند و بر روی آن قابل اجرا است.

ب) اجزای یک *Toolchain* برای این کار کدام اند و هریک چه نقشی در تبدیل کد *C++* روی *Host* به یک باینتری قابل اجرا روی *Target* دارند؟ همچنین توضیح دهید چرا با بزرگتر شدن پروژه، ابزار های *build* مانند *Make, CMake* اهمیت پیدا میکنند.

برای اینکه کد متنی *cpp* را به یک فایل باینتری ای که روی *ARM* ران بشود تبدیل بکنیم نیاز به *Cross-Toolchain* داریم که اجزای آن شامل

. *Compiler*: کد *c++* را به کد های اسمیبی مخصوص پردازنده *ARM* تبدیل کند

. *Assembler*: کد های اسمبلی را به کدهای باینری تبدیل کند و فایل *(object) o.* بسازد.

. *Linker*: فایل های *o.* مختلف را به هم متصل میکند و با اتصالشان به لایبرری ها یک فایل اجرایی میسازد.

. *Libraries*: کد های آماده *Target* مثل کتابخانه استاندارد *cpp* برای *arm* که لینکر این کد ها را به برنامه ما متصل میکند، این کتابخانه ها را باید بر روی *Host* داشته باشیم.

. *Binultils* ابزار های باینری جانبی برای بررسی، دیباگ و دستکاری فایل های باینری

با بزرگتر شدن پروژه و زیاد شدن فایل‌های *cpp* و وابستگی بین آن‌ها، به مشکلاتی نظیر طولانی شدن زمان کامپایل و پیچیدگی مدیریت فرآیند *Build* برمی‌خوریم. ابزارهایی مانند ***Make*** مشکل زمان را حل می‌کنند؛ به این صورت که با استفاده از فایل *Makefile* ، تشخیص می‌دهند کدام فایل‌ها تغییر کرده‌اند و به جای کامپایل مجدد کل پروژه، فقط فایل‌های تغییر یافته را کامپایل می‌کنند. از طرف دیگر، برای حل مشکل پیچیدگی مدیریت مسیرها هنگام کراس‌کامپایل، از ***CMake*** استفاده می‌کنیم. با اینکار به جای درگیری با مسیرهای هاردکد شده که مشکل عدم تطبیق هنگام همکاری را دارد، با استفاده از *CMakeLists.txt* به صورت سطح بالا تعیین می‌کنیم که پروژه چه فایل‌هایی دارد و به چه کتابخانه‌هایی وابسته است. سپس *CMake* با استفاده از فایل‌های پیکربندی *Toolchain* ، مسیر دقیق کتابخانه‌های مربوط به *Target* را پیدا کرده و *Makefile* نهایی را به صورت خودکار و بدون خطا تولید می‌کند.

سوال3: ***Root File System*** و فایل سیستم ها در ***Embedded Linux***

الف) نقش این چند پوشه مهم در ساختار *Root File System* را توضیح دهید: */bin, /etc, /lib, /dev, /var, /sys, /proc* برای */proc* و */sys* توضیح دهید چه تفاوتی با پوشه های معمولی دارند.

پوشه */bin* محل قراربری فایل های احرایی و دستورات پایه ای مثل *cp/ls/echo* است

پوشه */etc* شامل فایل های *configuration* سیستم است و تنظیماا شبکه و اسکریپت های راه اندازی و اطلاعات کاربران در این پوشه ذخیره میشوند.

پوشه */lib* کتابخانه ها مثل کتابخانه *C* در آن قرار دارد که برنامه های موجود در */bin* برای اجرا به این کتابخانه ها نیاز دارند. همچنین ماژول های کرنل نیز در این پوشه قرار دارند.

پوشه */var* *variable data* ها در این پوشه قرار میگیرندو فایل هایی که حجمشان دائما در حال تغییر است، مانند *log* ها و داده های موقت برنامه ها.

پوشه */dev* فایل های *device nodes* در آن قرار میگیرند و رابط اصلی تعامل با سخت افزار ها است مانند پورت های سریال یا فلش ای که به سیستم وصل میشوند.

پوشه */proc* پنجره ای به درون کرنل است و اطلاعات لحظه ای سیستم و پردازنده و وضعیت حافظه را شامل میشود

پوشه */sys* ساختا درختی سخت افزار ها و درایور های شناسایی شده توسط سیستم را شامل میشود.

پوشه های */proc, /sys* بر خلاف سایر دایرکتوری ها، روی حافظه فیزیکی وجود ندارند و *virtual file system* هعستند و روی *RAM* مستقیما ایجاد میشوند و توسط کرنل ساخته میشوند، محتوی این دو پوشه *volatile* میباشند و با هر بار راه اندازی سیستم از نو ساخته نمیشوند و با قطع برق کاملا از بین میروند.

ب) فایل سیستم های *ext4* و *SquashFS* و *tmpfs* را از نظر کاربرد در *Linux Embedded* مقایسه کنید و توضیح دهید هر کدام برای چه نوع داده با بخشی از سیستم مناسب ترند. در مقایسه خود ویژگی های تعیین کننده از جمله رفتار پس از قطع برق و اثر نوشتن مکرر بر حافظه فلش را در نظر بگیرید.

. *ext4*: قابلیت نوشتن و خواندن را داراست. به صورت مداوم تغییرات را مینوسد و بنابریان استهلاک آن به شدت بالا میرود. همچنین با قطع هنگام نوشتن احتمال *corruption* داده ها را داریم. همچنین این فایل سیستم *non-volatile* است. به دلیل این ویژگی ها برا پارتیشن های کوچک که نیاز به ذخیره پایدار دارند مناسب هستند مثل */data*

. *SquashFS*: قابلیت خواندن را فقط دارد *(Read-Only)* و فشرده است. بدلیل نبودن قابلیت نوشتن، *wear* ندارد و در برابر قطع شدن ناگهانی برق کاملاً مقاوم است و هیچ فایلی خراب نمی‌شود. این فایل‌سیستم نیز *Non-Volatile* است و داده‌های سیستم را برای همیشه ثابت نگه می‌دارد. به همین دلیل برای ذخیره فایل‌های اصلی و تغییرناپذیر سیستم‌عامل (مثل پوشه‌های */bin* و */lib*) ایده آل است.

. *Tmpfs*: به جای حافظه فلش، مستقیماً روی حافظه *RAM* اجرا می‌شود. سرعت بسیار بالایی دارد و چون روی فلش نوشته نمی‌شود، استهلاک آن صفر است. اما برخلاف دو مورد قبلی، این فایل‌سیستم کاملاً ***فرار (Volatile)*** است؛ یعنی به محض قطع شدن برق، تمامی اطلاعات آن از بین می‌رود. این ویژگی آن را برای پوشه‌هایی با نرخ نوشتن بالا که به ذخیره دائمی نیاز ندارند (مثل */var* برای لاگ‌ها و */tmp* برای فایل‌های موقت) بسیار مناسب می‌کند.

سوال4: *Device Tree* و *Device Tree Overlay*:

الف) چرا در *embedded Linux*، سخت افزار یا *Device Tree* توصیف میشود؟ همچنین توضیح دهید *Device Tree* معمولا چه نوع اطلاعاتی از سخت افزار را توصیف میکند.

در گذشته برای هر برد *ARM*، برنامه نویس ها تمام مشخصات سخت افزاری آن برد(مثل اینکه پورت سریال در چه آدرس حافظه ای قرار دارد) را مستقیما داخل کد های *C* کرنل لینوکس هاردکد میکردند، اینجوری لینوکس پر از هزاران فایل *C* برد های مختلف میشد که لاینوس تورووالدز از این خسته شد و *Device Tree* معرفی شد.

به جای نوشتن کد *C* یک فایل متنی ساده مینوسیم و در آن به کرنل میگوییم چه سخت افزار هایی روی برد داریم. کرنل موقع بوت شدن این فایل را میخواند و میفهمد روی چه بردی در حال اجرا است.

سخت افزار هایی مثل *USB* یا *PCI Express* ای که اصطلاحا *discoverable* هستند در *device tree* توصیف میشوند و نوعشان هم شناخته میشود. اما اکثر قطعات که *non-discoverable* هستند صرفا اطاعات *register*ها, *IRQs*, فرکانس کلاک ها و *pin Muxing* را دارند

ب) *Device Tree Overlay* چیست و در چه شرایطی مفید است؟ یک مثال مشخص بزنید که در آن یک *peripheral* با کمک *Overlay* بدون کامپایل مجدد کد کرنل فعلا و پیکربندی شود.

مکانیزکی است که به ما اجازه میدهد تغییرات، سخت افزاری جدید را به *Device Tree* اصلی سیستم اضافه کنیم بدون اینکه نیازی به تغییر و کامپایل مجدد *Device Tree* یا کرنل باشد. این مکانیزم در سیستم هایی که سخت افزار آنها به صورت ماژولار و متغیر است، بسیار حیاتی است که بجای ساختن ده ها فایل توصیف سخت افزار برای هر تکریب ممکن از *overlay* ها استفاده کنیم، برای مثال اگر قصد اضافه کردن یک سنسور جدید به سیستم بودیم، بدون استفاده از *DTO*، محبور بودیم فایل توصیف اصلی برد را در سورس کد کرنل پیدا کرده و *node* مربوط به سنسور را دستی اضافه کنیم و در نهایت فایل *DTB* اصلی یا حتی کل کرنل را از نوع کامپایل کرده و روی برد فلش کنیم.
اما با کمک *Overlay* تنها یک فایل متنی کوچک مینوسیم که در آن مشخص میکنیم که از کدام گذرگاه و چه آدرسی و شناساه ای قرار دارد که تبدیل به فاین باینری کوچکی تبدیل میشود. که توسط *bootloader* در هنگام بالا امدن سیستم خوانده میشود و پس از بوت شدن سیستم به صورت زنده به کرنل تزریق میشود. اینگونه سیستم *node* جدید را میشناسد و درایور مربوط به سنسور را بارگذاری میکند، در حالی که نیاز به کامپایل مجدد سیستم عالم نداشتیم.

سوال5: پیش بینی پذیری زمانی در *Linux* و سیستم های بی درنگ

الف) توضیح دهید چرا *Linux* استاندارد برای تضمین *hard real-time* مناسب نیست و منابع اصلی *latency* و *jutter* در آن کدام اند.

در طراحی لینوکس استاندارد، هدف بهینه سازی *throughput* و توزیع عادلانه منابع بوده است ولی در سیستم های *realtime* دغدغه اصلی سرعت میانگین/ *throughput* نیست و قطعیت زمانی و پاسخ دادن در *worstcase scenario* است و این با توزیع عادلانه منابع معمولا انجام نمیشود و لینوکس استاندارد این قطعیت را تضمین نمیکند.

منابع اصلی *Latency* و *Jitter*، بخش های زیر است:

. جاب های *non preemptive*: وقتی هسته در حال اجرای برخی کد های بحرانی است، برای جلوگیری از تداخل قفل هایی به نام *spinlock* استفاده میکند و *interrupt* ها را غیرفعال میکند. در این حالت، جاب های پراولویت هم اگر آماده اجرا باشند، هسته نمیتواند آن ها را متوقف کند و این باعث تاخیر های غیر قابل پیشبینی میشود.

. *interrupt handling*: روتین های هندل کردن در شرایط حاص میتوانند جاب های پر اولویت را به تاخیر بندازند.

. *Page fault*: جابجایی صفحات حافظه بین *RAM* و حافظه میتواند تاخیر های زیادی را ایجاد بکند.

ب) دو رویکرد زیر را از نظر رسیدن به رفتار زمانی قابل پیش بینی مقایسه کنید:

1. استفاده از *linux* همراه با *PREEMPT_RT*

2. واگذاری بخش حساس به زمان سیستم به یک *microcontroller* یا *RTOS* جداگانه و استفاده از *Linuz* برای بخش هایی که نیازمند پاسخ دهی با *deadline* سخت نیستند.

مزایا، محدودیت ها و *trade-off* هر رویکرد را توضیح دهید و بیان کنید در چه شرایطی هر کدام انتخاب مناسبتری است.

در رویکرد اول، تغییرات عمیقی در کرنل ایجاد میکنیم به این صورت که *spinlock* ها را به *mutex* تبدیل میکنیم و *interrupt* هل را در قالب *thread* هایی *schedulable* اجرا میکنیم، به این صورت لینوکس را به یک

سیستم عامل *real-time* تبدیل میکنیم. این روش بسیار یکپارچه و سریع است و همچنان یک سیستمعامل و *toolchain* داریم و برنامه ها میتوانند به راحتی از امکانات لینوکس نیز بهره ببرند ولی این روش *overhead* بالایی روی سیستم عامل قرار میدهد و تاخیر ها صرفا کراندار میشوند. همچنین به دلیل اشتراک حافظه *cache* بین پردازش های عادی و *real-time*، امکان تداخل وجود دارد.

در روش دوم کار های سنگین و غیر حساس روی لینوکس و کار های *real-time* روی یک پردازنده مجزا با سیستم عامل *RTOS* یا به صورت *Bare-metal* انجام میشود. این روش قطعیت زمانی خوبی دارد ولی پیچیدگی فوق العاده بالایی برای توسعه به همراه دارد، که دو معماری مختلف و دو سیستم عامل و دو *toolchain* مجزا را همزمان داریم و چالش های برقراری ارتباط بین پردازنده ها نیز وجود دارند و هزینه توسعه را بالا میبرند.

در سیستم های *soft real-time* تا *hard real-time* های تاخیر بزرگ استفاده از روش اول مناسب است ولی در سیستم های *hard real time* بحرانی که نمیازمند *response time* بسیار سریع هستند، استفاده از روش دوم مناسب است.

پخش عملی:

تمرین 1: سرویس محلی درخواست و پاسخ با *Unix Domain Socket*:

یک جفت برنامه به زبان *cpp* **در محیط لینوکس بنویسید: یک** *server* **و یک کلاینت. این دو برنامه باید از طریف یک** *unix domain socket* با یکدیگر ارتباط برقرار کنند و یک سرویس محلی ساده از نوع درخواست و پاسخ بسازند.

۱- برنام هی ***server*** یک ***Uni× domain socket*** روی مسیری که خودتان تعیین میکنید ایجاد کند، روی آن گوش بدهد و اتصالهای ورودی را بپذیرد. سرور باید پس از پاسخدادن به هر ***client*** فعال بماند و بتواند چند ***client*** را پشت سر هم سرویس دهد، نه فقط یک اتصال.

۲- یک پروتکل متنی ساده تعریف کنید که دو فرمان داشته باشد: فرمانی برای دریافت تاریخ و زمان فعلی سیستم از ***server*** و فرمانی برای بازگرداندن همان متنی که ***client*** ارسال کرده است. طراحی دقیق قالب فرمانها با خودتان است.

۳- برنامهی ***client*** فرمان و آرگومان احتمالی آن را هنگام اجرا در ترمینال دریافت کند، سپس به ***socket*** سرور متصل شود، درخواست را برای ***server*** بفرستد و پاسخ را از آن دریافت و چاپ کند و سپس خارج شود.

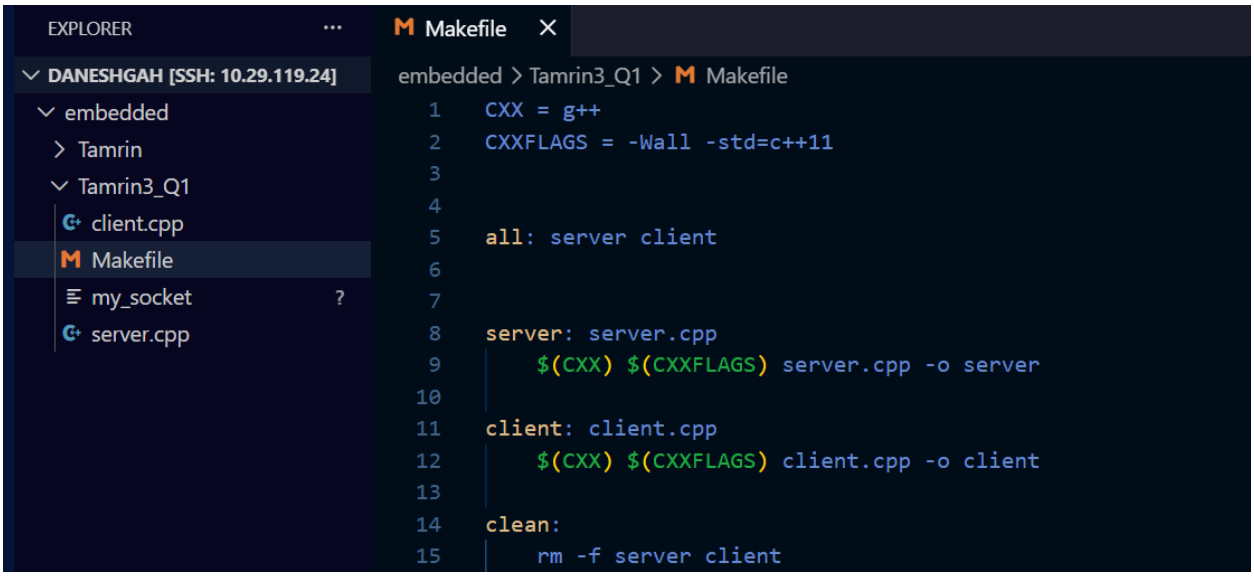
۴. یک ***Makefile*** بنویسید که هر دو برنامهی ***client*** و ***server*** را بسازد.

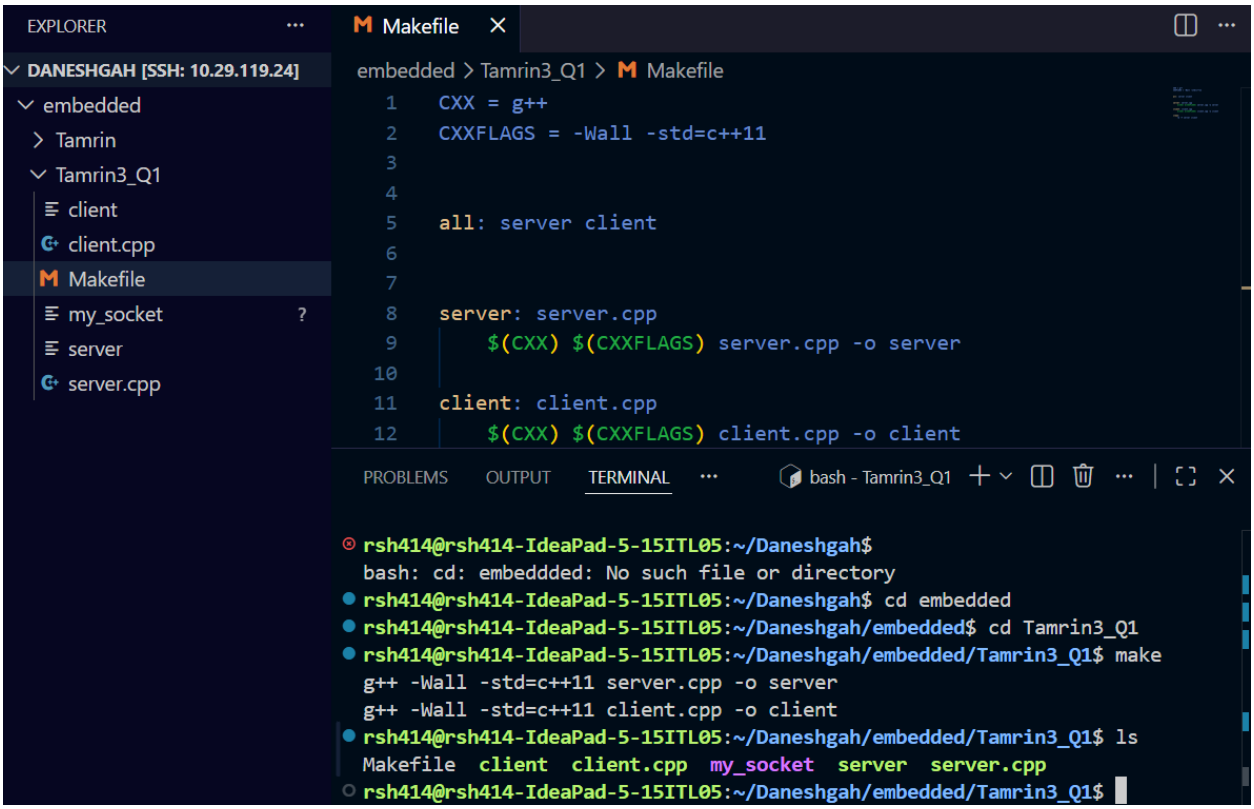
سه فایل ***server.cpp*** و ***client.cpp*** و ***Makefile*** را نوشتیم:

- ***server.cpp***: کد های مربوط به هسته اصلی سرویس دهنده
- ***client.cpp***: کد های مربوط به برنامه ی کاربر برای ارسال درخواست
- ***Makefile***: اسکریپت اتوماسیون برای کامپیال برنامه ها

MAKEFILE:

برای جلوگیری از کامپیال دستی مکرر از ***Makefile*** استفاده شده است. در این فایل کامپایلر ***g++*** با استاندارد ***c++11*** تنظیم شده است. با اجرای دستور ***make***، هر دو فایل اجرایی ساخته میشوند و با دستور ***make clean*** فایل های تولید شده پاک میوند.


EXPLORER
DANESHGAH [SSH: 10.29.119.24]
embedded
Tamrin
Tamrin3_Q1
client.cpp
Makefile
my_socket
server.cpp
Makefile
embedded > Tamrin3_Q1 > Makefile
1 CXX = g++
2 CXXFLAGS = -Wall -std=c++11
3
4
5 all: server client
6
7
8 server: server.cpp
9 $(CXX) $(CXXFLAGS) server.cpp -o server
10
11 client: client.cpp
12 $(CXX) $(CXXFLAGS) client.cpp -o client
13
14 clean:
15 rm -f server client


EXPLORER
DANESHGAH [SSH: 10.29.119.24]
embedded
Tamrin
Tamrin3_Q1
client
client.cpp
Makefile
my_socket
server
server.cpp
Makefile
embedded > Tamrin3_Q1 > Makefile
1 CXX = g++
2 CXXFLAGS = -Wall -std=c++11
3
4
5 all: server client
6
7
8 server: server.cpp
9 $(CXX) $(CXXFLAGS) server.cpp -o server
10
11 client: client.cpp
12 $(CXX) $(CXXFLAGS) client.cpp -o client
PROBLEMS
OUTPUT
TERMINAL
bash - Tamrin3_Q1
rsh414@rsh414-IdeaPad-5-15ITL05:~/Daneshgah$
bash: cd: embeddded: No such file or directory
rsh414@rsh414-IdeaPad-5-15ITL05:~/Daneshgah$ cd embedded
rsh414@rsh414-IdeaPad-5-15ITL05:~/Daneshgah/embedded$ cd Tamrin3_Q1
rsh414@rsh414-IdeaPad-5-15ITL05:~/Daneshgah/embedded/Tamrin3_Q1$ make
g++ -Wall -std=c++11 server.cpp -o server
g++ -Wall -std=c++11 client.cpp -o client
rsh414@rsh414-IdeaPad-5-15ITL05:~/Daneshgah/embedded/Tamrin3_Q1$ ls
Makefile client client.cpp my_socket server server.cpp
rsh414@rsh414-IdeaPad-5-15ITL05:~/Daneshgah/embedded/Tamrin3_Q1$

Server.cpp:

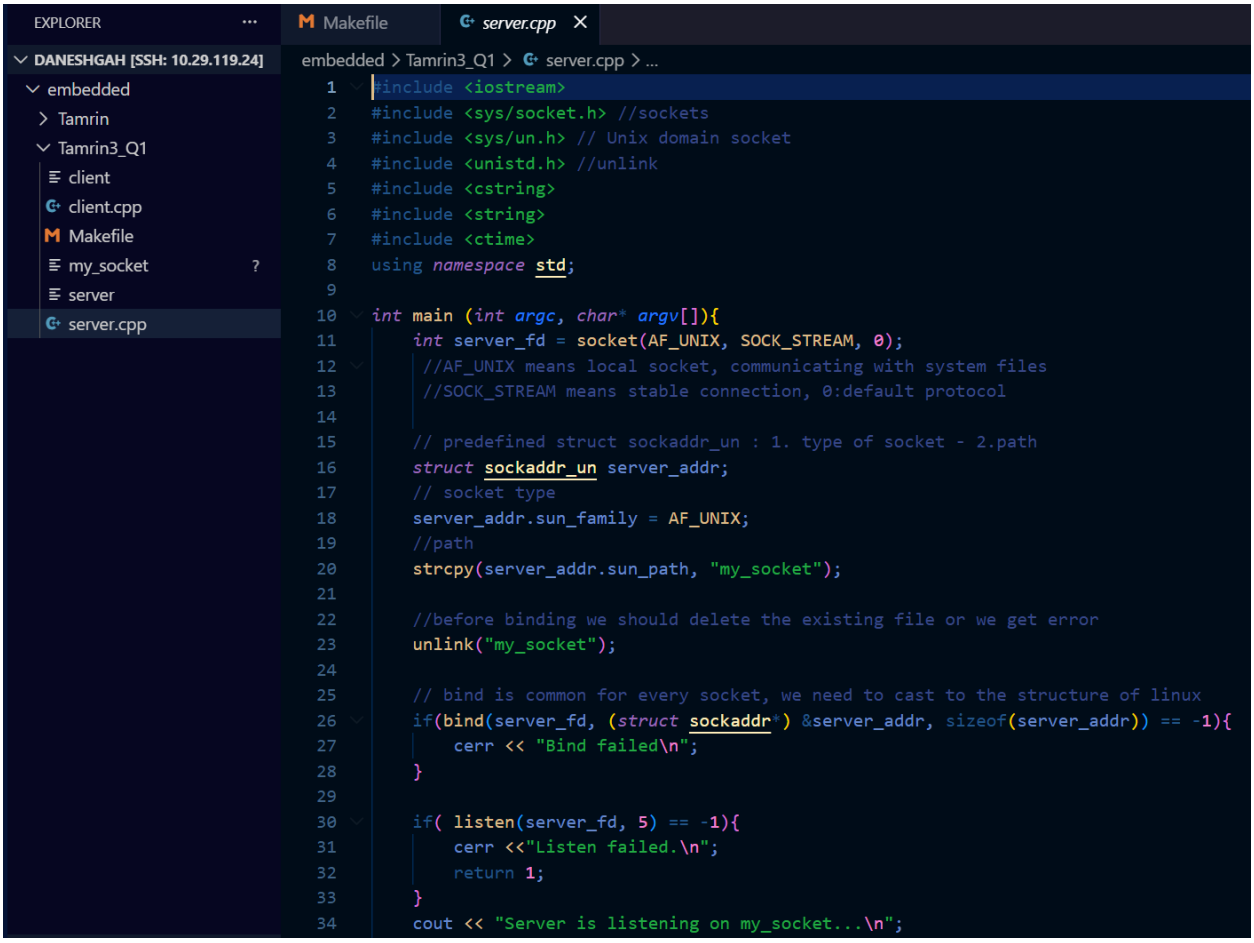
در این برنامه ، یک سوکت روی سیستم عامل درست میکنیم و آن را به یک ***file directory*** مقید میکنیم و سپس منتظر درخواست های ورودی گوش میکنیم.

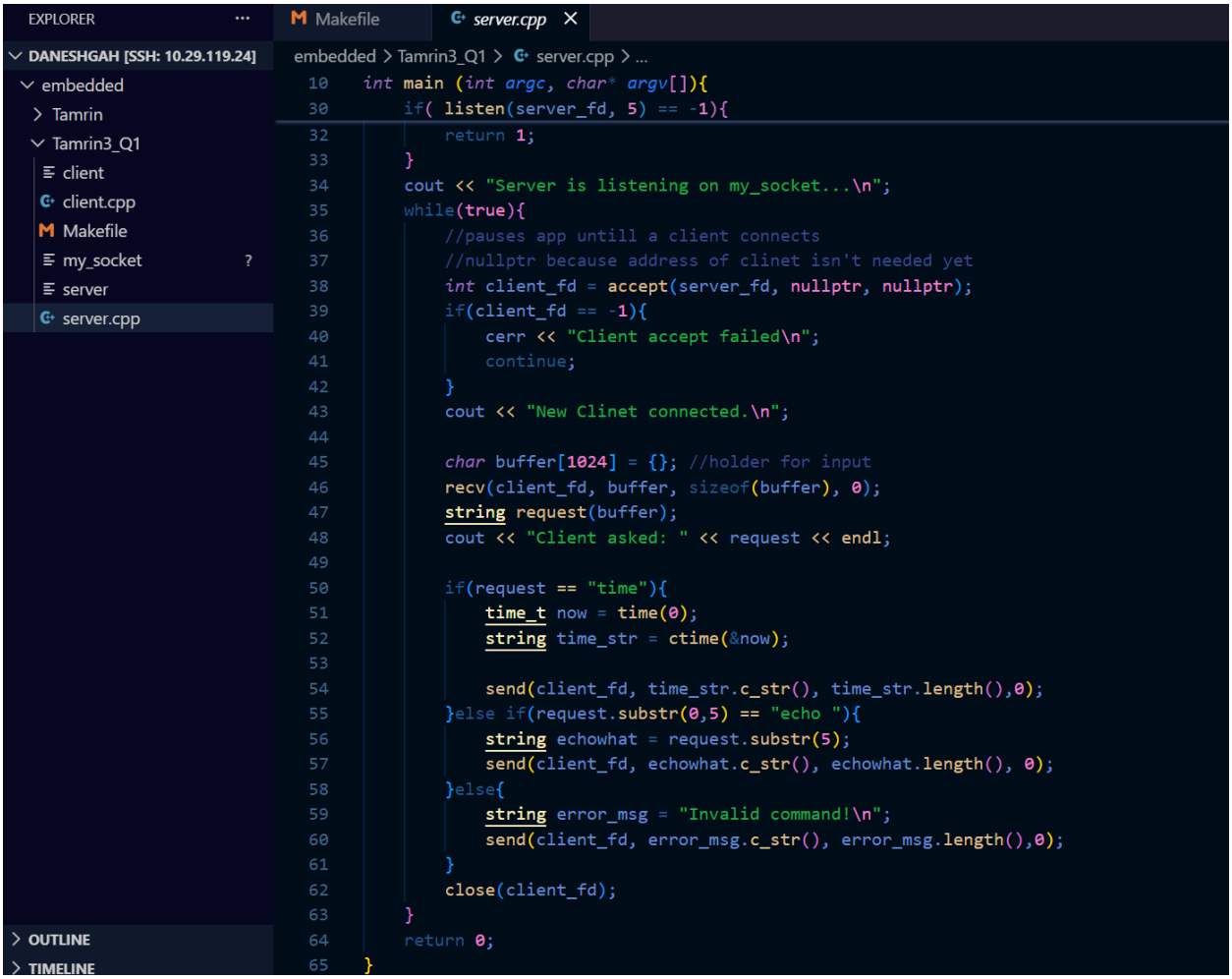
برای این کار، ابتدا با استفاده از ***socket(AF_UNIX, SOCK_STREAM, 0)*** یک سوکت محلی پایدار ایجاد میکنیم.

سپس مسیر سوکت را به صورت ***"my_socket"*** در ساختار ***sockjaddr_un*** تنظیم میکنیم.

همچنین از آنجایی که در صورت بسته شدن ناگهانی برنامه، فایل سوکت روی سیستم باقی میماند، پیش از ***bind*** کردن، از تابع ***unlink("my_socket")*** برای پاک سازی مسیر استفاده کردیم.سپس سوکن را در حالت ***listen*** قرار دادیم و درونه یک حلقه بینهایت ***while(true)*** تابع ***accept*** فراخوانی شد تا این تابع برنامه ی سرور را متوقف میکند تا کلاینت متصل شود.

پس از اتصال ***client*** با دریافت پیامش با تابع ***recv()*** پیام را به استرینگ تبدیل میکنیم و بررسی میکنیم که اگر پیام دریافتی ***"time"*** بودی، زمان سیستم را با با ***send()*** برمیگردانیم و در صورتی که ***"echo ..."*** بود بخش ابتدایی را حذف کنیم و مابقی استرینگ را به کلاینت بر میگردانیم، همچنین ر صورت نامعتبر بودنه دستور، ***Invalid command!*** ارسال می شود.



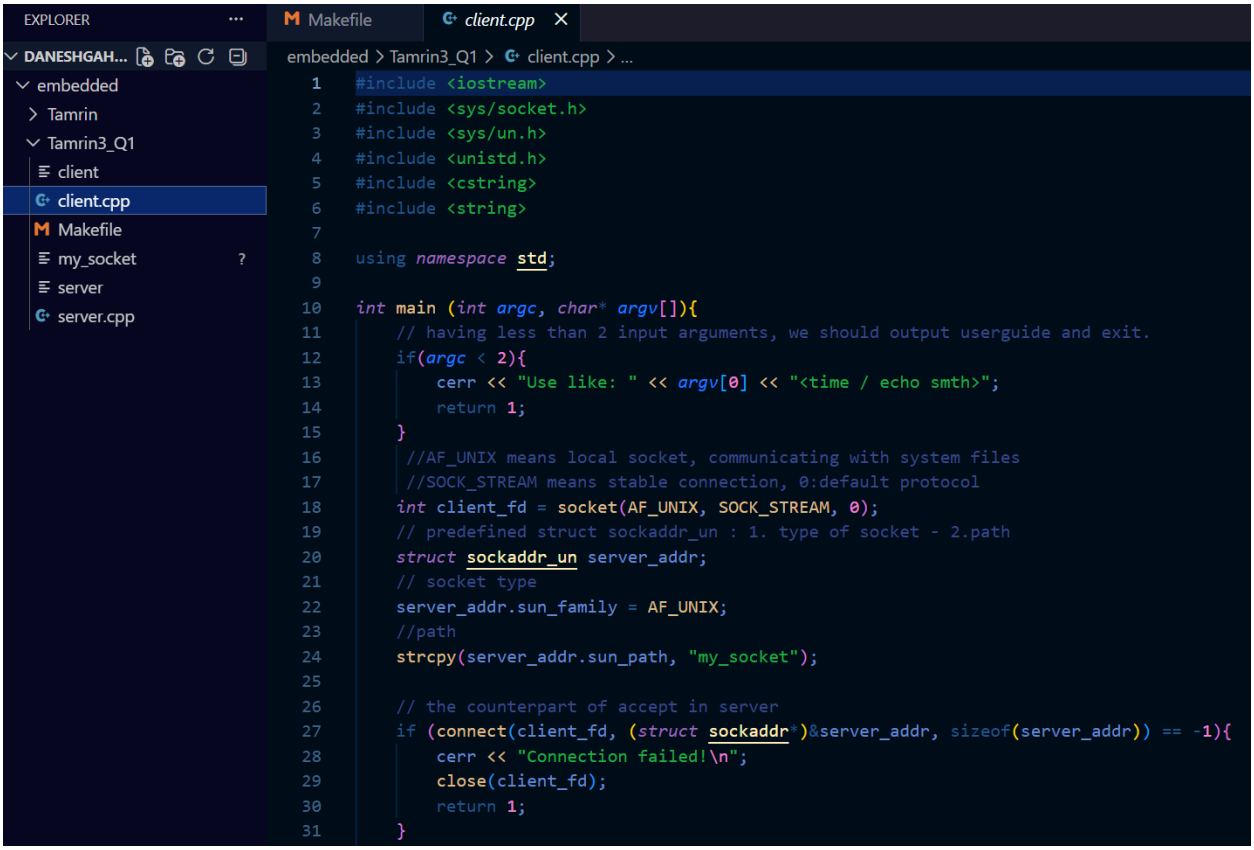


Client.cpp:

در برنامه ی کلاینت، باید دستورات را به صورت ***comman line*** به همراه آرگومان هایشان دریافت بکنیم و از طریق اتصال به سوکت سرور آن ها را پردازش کنیم.

ابتدا با ببرسی ***argc*** که تعداد ارگومان های دریافتی را نشان میدهد، اطمینان حاصل میکنیم که کاربر دستور ای همراه با آرگومان ها در ترمینال وارد کرده است. در غیر این صورت راهنمایی برای استفاده برای کاربر چاپ میکنیم.

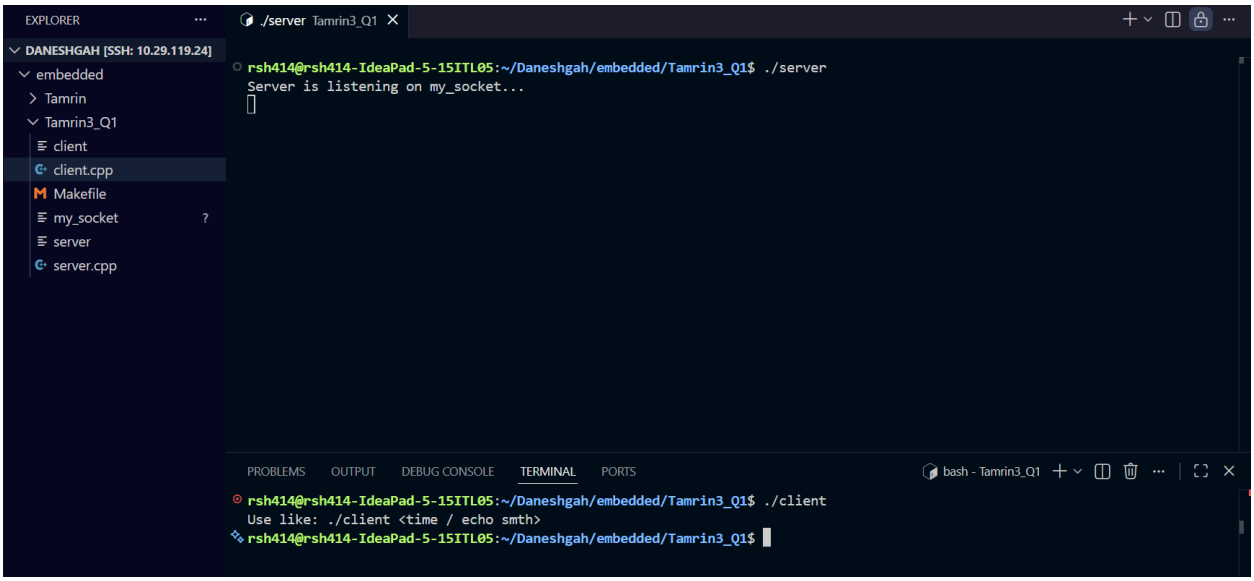
سپس سوکت ای از نوع ***AF_UNIX*** ساختیم و با تابع ***connect*** به آدرس ***"my_socket"*** متصل کردیم ، سپس با تبدیل آرگومان های ورودی با تسافده از یک حلقه ***for*** روی ***argv*** کلمات را به هم وصل میکنیم و بینشان فاصله میگذاریم تا یک استرینگ واحد ***request*** تشکیل بشود. سپس پیام ساخته شده را با ***send*** به سرور ارسال میکنهیم و کلاینت با تابع ***recv()*** منتظر دریافت پاسخ میماند تا آن را دریفات کند و سپس در تیمرنال چاپ میکنیم. پس از این سوکت از طرف کلاینت بسته میشود و کلاینت خاتمه میابد، در حالی که سرور برای کلاینت بعدی روشن میماند.


EXPLORER
DANESHGAH...
embedded
Tamrin
Tamrin3_Q1
client
client.cpp
Makefile
my_socket
server
server.cpp
Makefile
client.cpp
embedded > Tamrin3_Q1 > client.cpp > ...
#include <iostream>
#include <sys/socket.h>
#include <sys/un.h>
#include <unistd.h>
#include <cstring>
#include <string>
using namespace std;
int main (int argc, char* argv[]){
// having less than 2 input arguments, we should output userguide and exit.
if(argc < 2){
cerr << "Use like: " << argv[0] << "<time / echo smth>";
return 1;
}
//AF_UNIX means local socket, communicating with system files
//SOCK_STREAM means stable connection, 0:default protocol
int client_fd = socket(AF_UNIX, SOCK_STREAM, 0);
// predefined struct sockaddr_un : 1. type of socket - 2.path
struct sockaddr_un server_addr;
// socket type
server_addr.sun_family = AF_UNIX;
//path
strcpy(server_addr.sun_path, "my_socket");
// the counterpart of accept in server
if (connect(client_fd, (struct sockaddr*)&server_addr, sizeof(server_addr)) == -1){
cerr << "Connection failed!\n";
close(client_fd);
return 1;
}

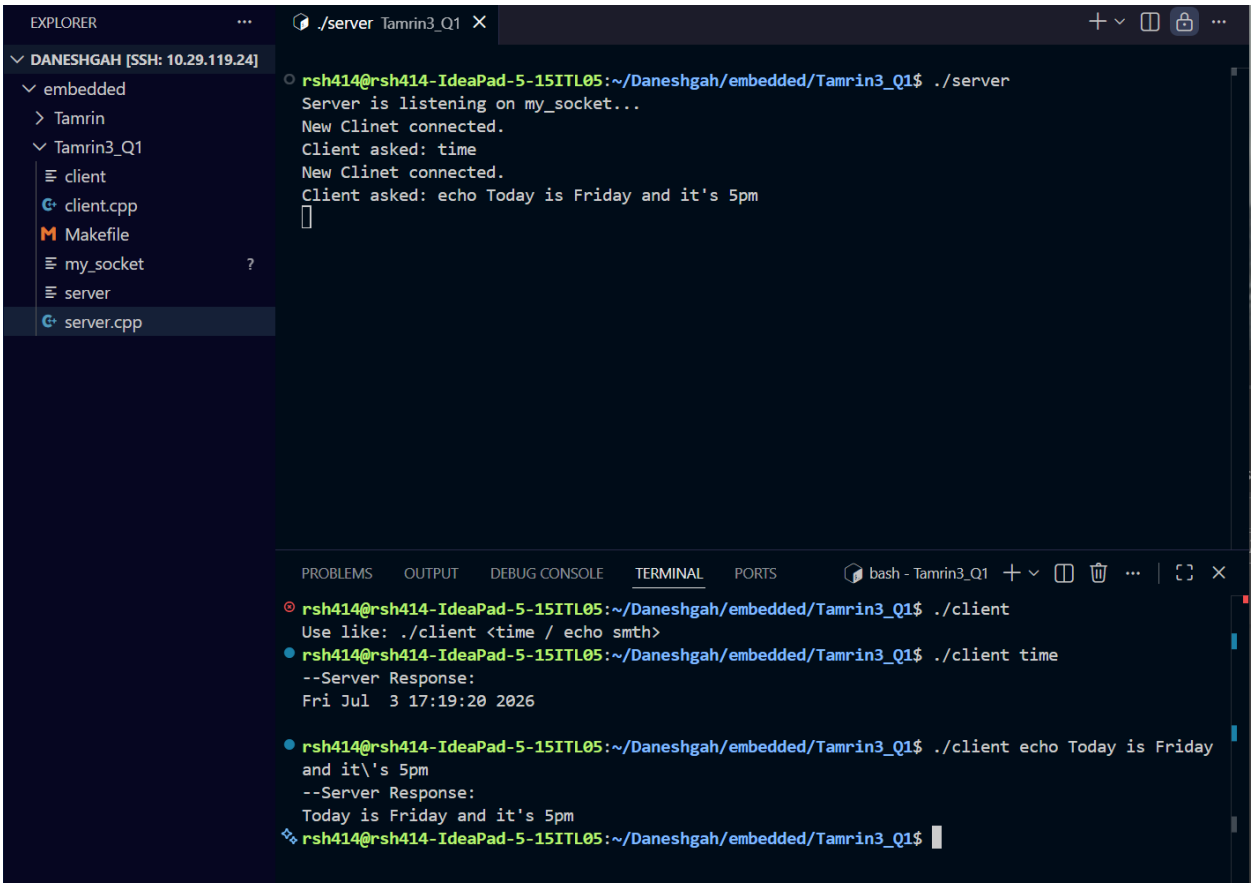

EXPLORER
DANESHGAH [SSH: 10.29.119.24]
embedded
Tamrin
Tamrin3_Q1
client
client.cpp
Makefile
my_socket
server
server.cpp
Makefile
client.cpp
embedded > Tamrin3_Q1 > client.cpp > main(int, char * [])
int main (int argc, char* argv[]){
if (connect(client_fd, (struct sockaddr*)&server_addr, sizeof(server_addr)) == -1){
// make char array to a string named request
string request = "";
for(int i = 1; i<argc ; i++){
request += argv[i];
if(i < argc - 1)
request += " ";
}
//send request
send(client_fd, request.c_str(), request.length(), 0);
//holder for answer of the server
char buffer[1024] = {0};
recv(client_fd, buffer, sizeof(buffer),0);
//server response printing
cout << "--Server Response: \n" << buffer << endl;
//closing client connection
close(client_fd);
return 0;
}

نتایج اتصال:

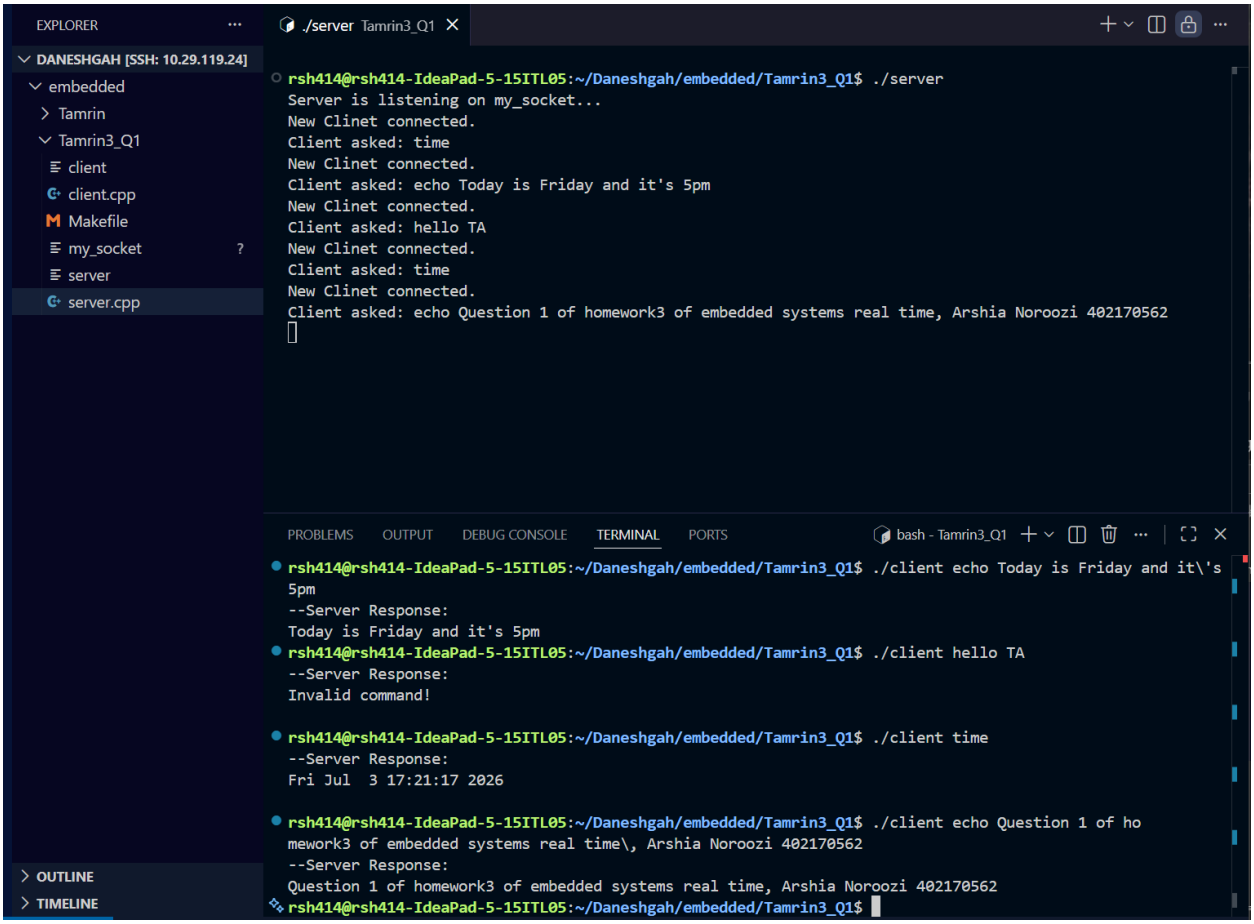
. برای دستور خالی راهنما ارسال میکنیم:



. دستور ***time*** و سپس ***echo*** پیام کلاینت:



. دستور ***Invalid*** و اکتیو ماندن سرور و دریافت کلاینت ها پشت سر هم:



تمرین 1: پایشگر محلی سیستم *(Local System Monitor)*

بخش اول

کد ها با پوشه های ***src*** و***include*** سازماندهی شد و فایل های ***Makefile*** و ***CMakeLists.txt*** برای ساخت برنامه نوشتیم:

کد ها را به دو بخش تقسیم کردیم و یکی را در پوشه ***src*** با اسم ***main.cpp*** نوشتیم و منطق اصلی توابع را در اون اوردیم و دیگری را در پوشه ***include*** که شامل ***procwatch.hpp*** است اوردیم که در آن تعریف استراکت ها و اینکلود ها رو گذاشتیم همچنین ساختار کلی توابع را در آن اوردیم.

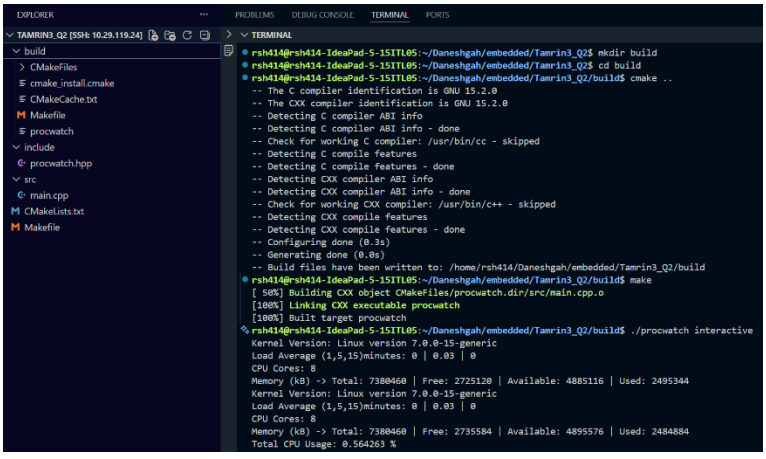
برای کامپایل این برنامه از هر دو سیستم ***Make , Cmake*** استفاده شد تا قابلیت حمل و ساختاریافتگی برنامه تضمین بشود.

CMake:

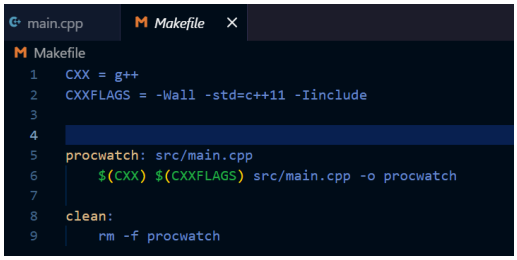
main.cpp | CMakeLists.txt

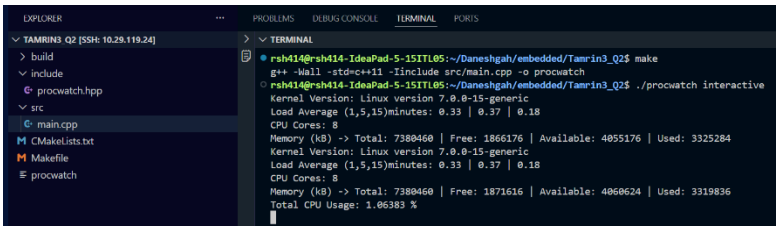
CMakeLists.txt

```
1  cmake_minimum_required(VERSION 3.10)
2
3  project(procwatch)
4
5  include_directories(include)
6
7  add_executable(procwatch src/main.cpp)
```



Make:





اطلاعات سیستم را با خواندن فایل های متنی در سیستم فایل محازی ***/proc*** بدست اوردیم:

. نسخه کرنل را از خواندن ***/proc/version*** بدست اوردیم و قسمت اول آن را استخراج کردیم.

. ***load average***: را با خواندن سسه عدد اول از ***/proc/loadavg*** که نمیانگر میانگین بار در 1 دقیقه 5 دقیقه و 15 دقیق گذشته هستند بدست آوردیم.

. ***processor count*** را با شمارش خروجی حاصل از ***proc/cpuinfo*** بدست اوریدم.

. وضعیت حافظه را هم با خواندن مقادیر ***MemFree***،***MemTotal*** و ***MemAvailable*** از ***/proc/meminfo*** و با تفریق کل از خالی***Memused*** را بدست آوریدم.

این اطلاعات را با در تابع ***printStatic()*** به صورت زیر در ترمینال چاپ میکنیم.

```
Kernel Version: Linux version 7.0.0-15-generic
Load Average (1,5,15)minutes: 0.31 | 0.37 | 0.18
CPU Cores: 8
Memory (kB) -> Total: 7380460 | Free: 1882212 | Available: 4071216 | Used: 3309244
```

برای محاسبه درصد مصرف ***cpu*** کل سیستم، چون لینوکس میزان مصرف پردازنده را به صورت درصدی و لحظه ای نگه داری نمیکند و مقادیر را به صورت ***counter***هایی ***cumulative*** در ***/proc/stat*** نگه داری میکند، که نشان میدهد از زمان روشن شدن سیستم تاکنون، پردازنده چه مقدار زمان بر حسب ***tick*** در حالت های مختلف صرف کرده است، نمیتوانیم با یک بار خواند درصد را به دست بیاریم، بنابراین برای اینکار در بازه های زمانی مشخص که ما دو ثانیه قرارش دادیم، فایل را میخوانیم و اختلاف آن با دفعه ی قبل را محاسبه میکنیم و به عنوان درصد گزارش میکنیم.

برای محاسبه ی درصد مصرف ***cpu*** ، اطلاعات زیر را از ***/proc/stat*** میخوانیم:

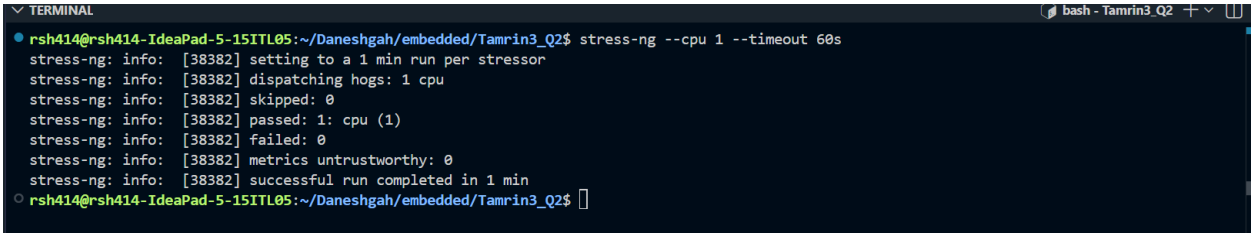
در خط اول این فایل 10 مقدار آمده که هر کدام به ترتیب به صورت زیر است:

1. ***user*** : زمان اجرای فرایند های ***user space***
2. ***nice*** : زمان اجرای فرآنید های کاربری با اولیوت پایین
3. ***system***: زمان اجرای کد های کرنل
4. ***idle***: زمان بیکاری
5. ***iowait***: زمان انتظار پردازنده برای اتمام عملیات دیسک
6. و7. ***irq, softirq***: زمان صرف شده برای مدیریت وقفه های سخت افزاری و نرم افزاری
8. و9و10. ***quest_nice, guest, steal***: زمان های مربوط به محیط های مجازی سازی:

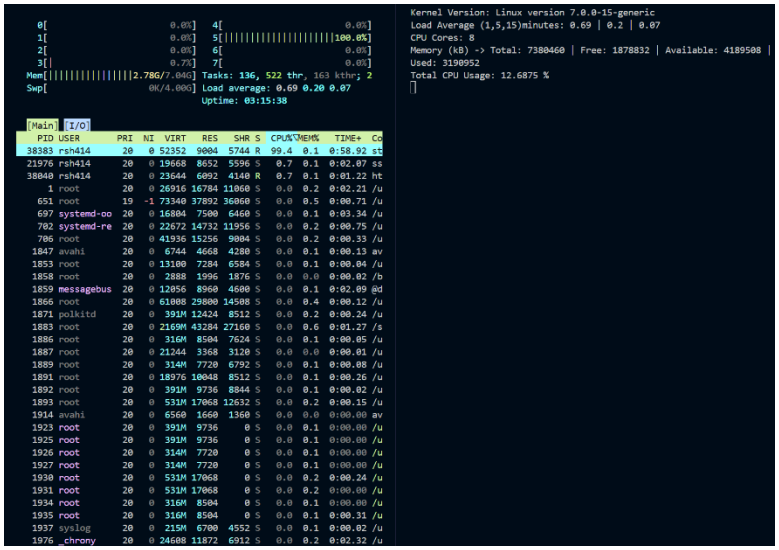
کافیست زمان های بیکاری را تقسیم بر زمان کل کنیم.

$$CPU\ usage = \frac{\Delta T_{total} - \Delta T_{idle}}{\Delta \mathrm{T_{total}}}$$

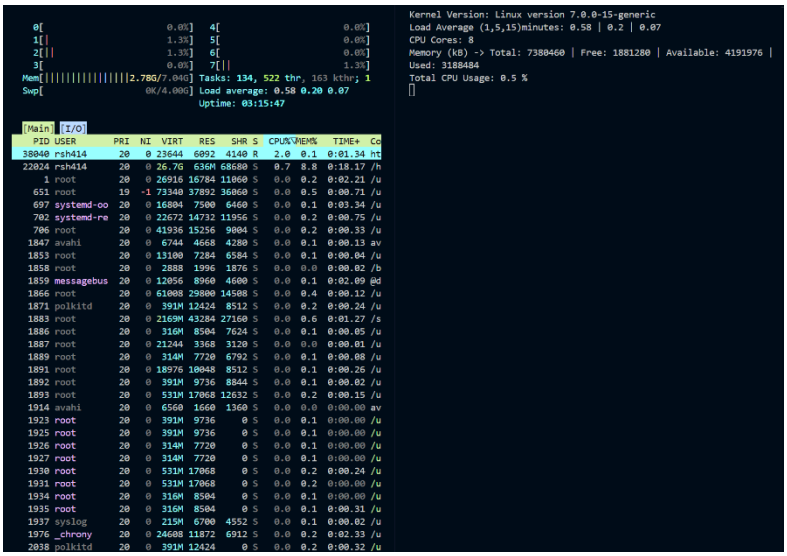
تست این بخش با ***stress-ng***: ابزار ***stress-ng*** را با ***apt*** نصب کردیم و سپس با آن یک بار پردازشی سنگین به مدت 60 ثانیه روی هسته های پردازنده انداختیم و سپس همزمان خروجی برنامه ی ***ptocwatch*** را با ایزار ***htop*** مقایسه کردیم:



مقایسه هنگام اعمال لود:



مقایسه پس از اتمام لود:



مقایسه کردیم که ***load_average*** مان دقیقا برابر است و در ***memory*** هم مشاهده میکنیم که اعداد کیلوبایتمان به گیگابایت تبدیل شتده اند و برابر اند و زیر لود هم ابزار ***htop*** یکی از هسته ها را 100 درصد نشان داده است که ابزار ما نیز درصد 12.68 درصد را نشان میدهد که میانگین هسته های مختلف است و صحت ابزارمان را نشان میدهد.

بخش دوم : پردازش های پر مصرف *top-n*

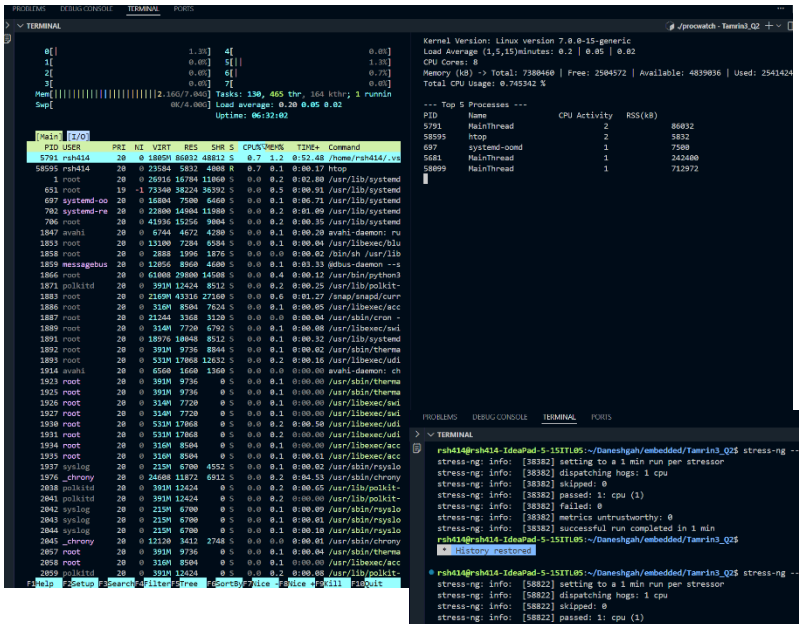
برای پیدا کردن پردازش های در حال اجرا، دایرکتوری */proc* را بررسی کردیم و آنهایی که صرفا عدد هستند که این عدد همان *PID* آنها است، اطلاعاتشان را برداشتیم یعنی نام پردازش که با خواندن *proc/<pid>/comm* میخوانیم و میزان حافظه *rss* که با خواندن *proc/<pid>/status* و کنار *VmRSS* نوشته میشد میخوانیم. همچنین از آنجایی که واحد پیشفرض کیلوبایت است، این واحد را هم یادداشت میکنیم.

همچنین برای محسابه میزان فعالیت پردازنده ی هر پردازش، در همان دایرکوتری *status* به مقادیر چهاردهم و پانزدهم نیاز داریم که *utime* یعنی زمان اجرا در *user space* و *stime* زمان اجرا در *kernel sapcve* هستند، این مقادیر را استخراج کردیم و مجموع آن ها زمان پردازش را شنان میدهند که با تفاضل این دو مقدار در دو خواندن متوالی، میزان فعالیت پردازنده را نشان میدهد. برای تبدیل این میزان فعالیت به درصد واقعی، باید این احتلاق را بر اختلاف زمانی کل سیستم دار همین بازه ی زمانی تقسیم کنیم. در واقع باید ببینییم پردازشمان چند واحد از کل واحد های زمانی سپری شده را به خود اختصاص داده است. که این زمان را در بخش اول به دست آوریدم و آن را در متغیر *totalDiff* نوشتیم با تقسیم بر آن میتوانیم میزان فعالیت پردازنده ی هر پردازش را به صورت درصدی حساب کنیم.

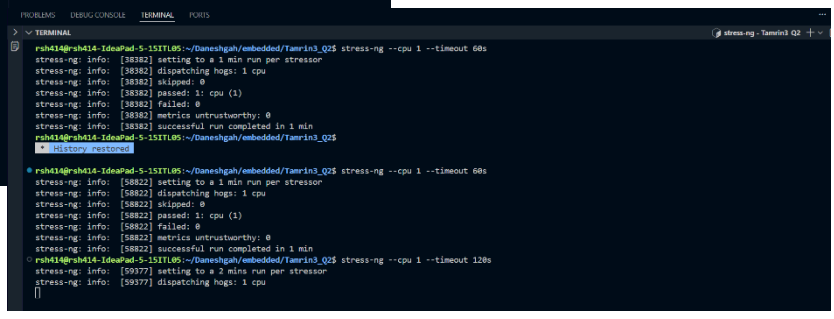
برای مقاومت در برابر خطا ها نیز، می بایست در برابر پردازش هایی که دقیقا در لحظه استخراج اطلاعات خاتمه میابند و فایل های آن پاک میشوند مقاوم باشیم، برای این کار برای جلوگیری از کرش کردن برنامه، پیش از خواندن مختوای فایل ها، از باز بودن و در دسترس بودنشان با *file.is_open()* اطمینان حاصل کردیم. درصورت بسته شدن یا عدم دسترسی بدون خطا آن *PID* را نادیده میگیریم.

تست این بخش:

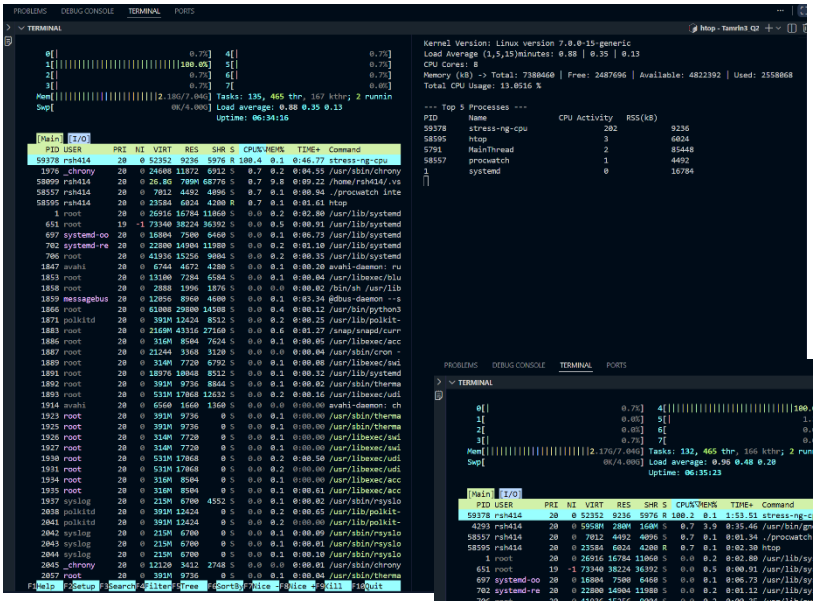
قبل استرس:

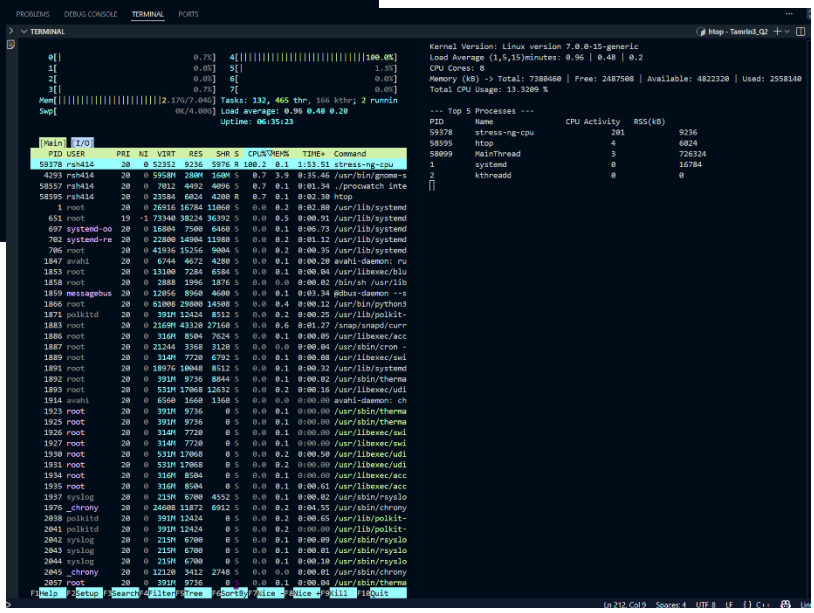


stressng



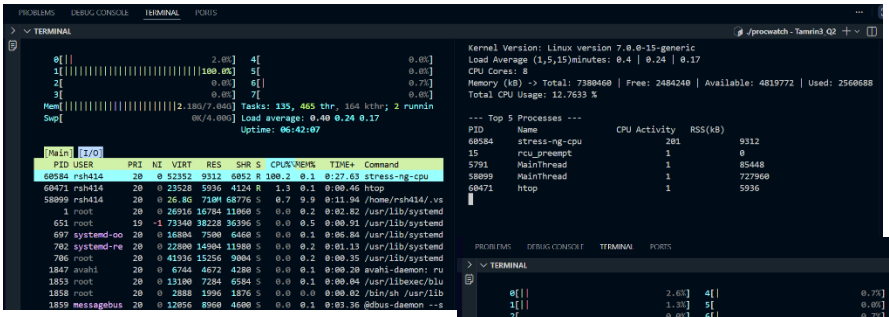
اواسط تست:

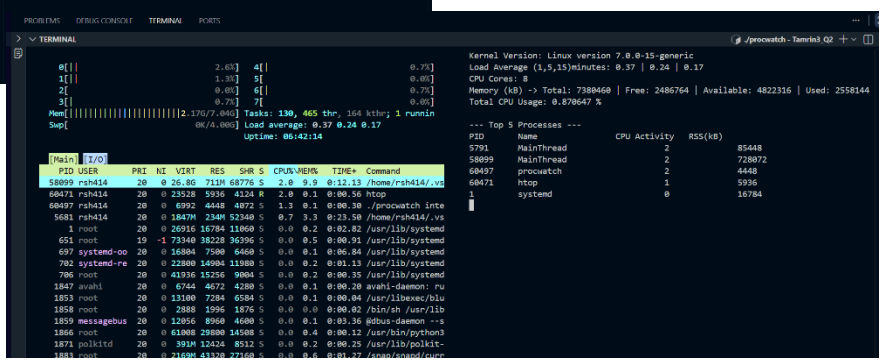




بعد از تست:

برنامه با تمام شدن پردازش، دچار توقف یا کرش نشده





بخش سوم:

در این بخش برنامه را به یک سرویس استاندارد لینوکسی که در پس زمینه کار میکند تبدیل میکنیم برای این کار:

پارامتر های اجرایی را از فایلی به نام ***config*** که در پوشه ***src*** دارد ست میکنیم، که این پارامتر ها شامل ***INTERVAL*** که بازه زمانی بروزرسانی است و ***N*** که تعداد پردازش های پر مصرف اول که چاپ میشود است و***CRASH_TEST*** که پرچم مربوط به ***systemd*** است .

سپس خروجیمان را از ترمینال خارج کردیم تا با استفاده از کتابخانه ***<syslog.h>*** خروجی را با ***openlog ,*** ***syslog*** در لاگ سیستم عامل ثبت کنیم و با ***journalctl*** آنها را در ادامه بخوانیم.

این بار هم محاسبه ی میزان فعالیت پردازنده را مانند بخش های قبل حساب کردیم.

همنین برای مدیریت سیگنال ها و توقف درست، از کتابخانه ***<csignal>*** استفاده کردیم و با نوشتن تابعی به نام ***signalHandler*** هنگامی که سیگنال ***SIGTERM*** از طریف سیستم عامل ارسال میشود، ***keeprunning*** را ***false*** میکنیم تا برنامه حافظه های اشغالی را رها کند و با نوشتن پیام ***Service stopped gracefully.*** برنامه را ببندد.

همچنین برای اطمینان و اینکه قابلیت بازیابی خودکار داشته باشید ***(Restart=on-failure)***
فایل***procwatch.service*** را درست کردیم و برای تست آن پارامتر ***CRASH_TEST*** را تعریف کردیم تا با فعال کردنش برنامه عمدا خطایی درست کند و عددی غیر 0 ریترن کند، با این کار ***systemd*** تریگر میشود و بخاطر ***RestartSec = 5s*** سرویس 5 ثانیه بعد خودکار دوباره راه اندازی میشود.

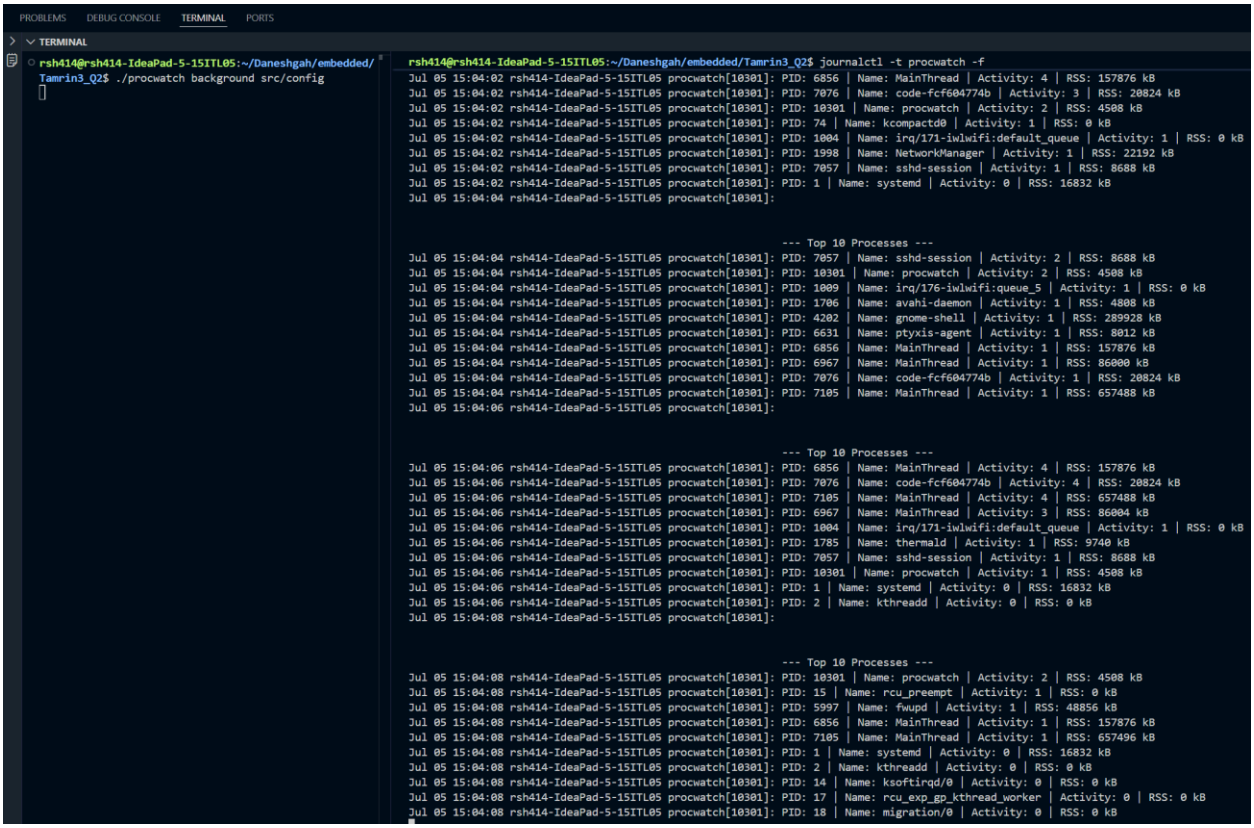
برای اینکه ***systemd***بتواند مدیریت برنامه ما را به دست بگیرد و آن را به عنوان یک سرویس بشناسد، فایل اجرایی ***procwatch***را به مسیر استاندارد برنامه‌های لینوکس یعنی ***/usr/local/bin/***و فایل تنظیمات را به مسیر ***/etc/procwatch.conf***منتقل کردیم. سپس فایل پیکربندی ***procwatch.service***را در دایرکتوری ***/etc/systemd/system/***قرار دادیم. در نهایت، با اجرای دستور ***systemctl daemon-reload***، سرویس

جدید را به سیستم‌عامل شناساندیم و با دستور ***systemctl start procwatch*** اجرای برنامه را آغاز کردیم تا سیستم‌عامل به صورت کامل روی چرخه حیات آن نظارت داشته باشد.

```
rsh414@rsh414-IdeaPad-5-15ITL05:~/Daneshgah/embedded/Tamrin3_Q2$ systemctl status procwatch
● procwatch.service - Procwatch System Monitor
     Loaded: loaded (/etc/systemd/system/procwatch.service; disabled; preset: enabled)
     Active: active (running) since Sun 2026-07-05 15:50:33 +0330; 16s ago
 Invocation: 53b90b7f2076402997375012d363a8fa
   Main PID: 16326 (procwatch)
      Tasks: 1 (limit: 8593)
     Memory: 880K (peak: 1.9M)
        CPU: 134ms
     CGroup: /system.slice/procwatch.service
             └─16326 /usr/local/bin/procwatch background /etc/procwatch.conf

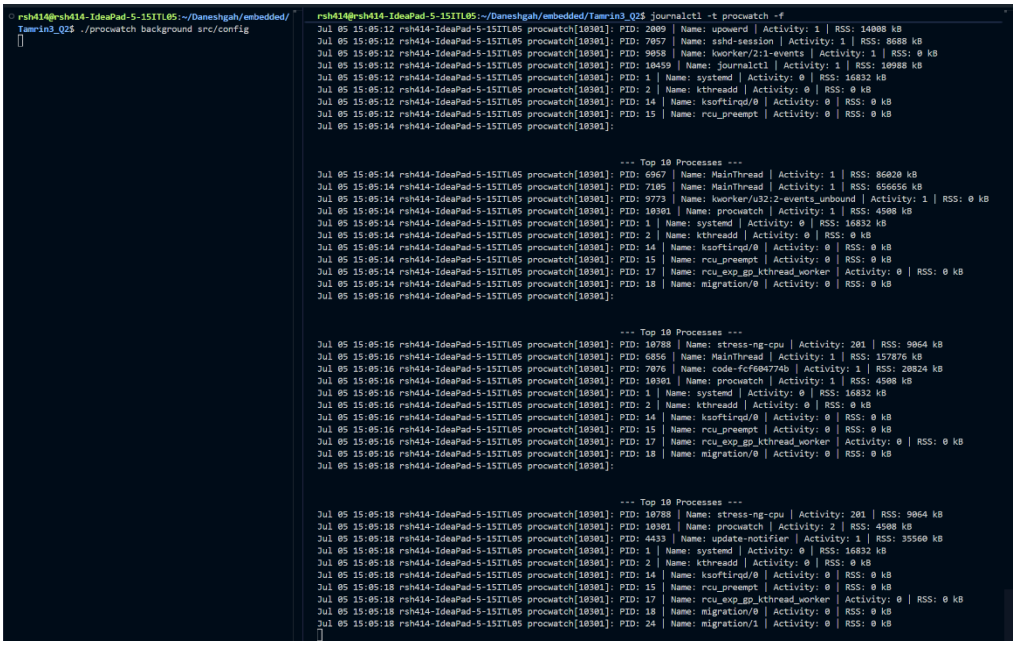
Jul 05 15:50:49 rsh414-IdeaPad-5-15ITL05 procwatch[16326]: PID: 6967 | Name: MainThread | Activity: 4 | RSS: 87048 kB
Jul 05 15:50:49 rsh414-IdeaPad-5-15ITL05 procwatch[16326]: PID: 7105 | Name: MainThread | Activity: 3 | RSS: 691892 kB
Jul 05 15:50:49 rsh414-IdeaPad-5-15ITL05 procwatch[16326]: PID: 16326 | Name: procwatch | Activity: 2 | RSS: 4480 kB
Jul 05 15:50:49 rsh414-IdeaPad-5-15ITL05 procwatch[16326]: PID: 4202 | Name: gnome-shell | Activity: 1 | RSS: 290084 kB
Jul 05 15:50:49 rsh414-IdeaPad-5-15ITL05 procwatch[16326]: PID: 6856 | Name: MainThread | Activity: 1 | RSS: 159072 kB
Jul 05 15:50:49 rsh414-IdeaPad-5-15ITL05 procwatch[16326]: PID: 7076 | Name: code-fcf604774b | Activity: 1 | RSS: 20828 kB
Jul 05 15:50:49 rsh414-IdeaPad-5-15ITL05 procwatch[16326]: PID: 9850 | Name: bash | Activity: 1 | RSS: 5912 kB
Jul 05 15:50:49 rsh414-IdeaPad-5-15ITL05 procwatch[16326]: PID: 1 | Name: systemd | Activity: 0 | RSS: 17728 kB
Jul 05 15:50:49 rsh414-IdeaPad-5-15ITL05 procwatch[16326]: PID: 2 | Name: kthreadd | Activity: 0 | RSS: 0 kB
Jul 05 15:50:49 rsh414-IdeaPad-5-15ITL05 procwatch[16326]: PID: 14 | Name: ksoftirqd/0 | Activity: 0 | RSS: 0 kB
rsh414@rsh414-IdeaPad-5-15ITL05:~/Daneshgah/embedded/Tamrin3_Q2$
```

بررسی وضعیت سرویس ***procwatch*** با دستور ***systemctl status*** که نشان می‌دهد برنامه با موفقیت در پس‌زمینه سیستم‌عامل در حال اجراست: وضعیت ***active (running)***



برنامه هر دو ثانیه ***N*** پراسس پر مصرف را در لاگ ذخیره میکند.

با ران کردن ***stress-ng*** این برنامه را در پراسس های پر مصرف میبینیم:



با زدن ***killall procwatch*** در ترمینالی دیگر سیگنال ***SIGTERM*** را به برنامه میدهیم:

```
rsh414@rsh414-IdeaPad-5-15ITL05:~/Daneshgah/embedded/Tamrin3_Q2$ stress-ng --cpu 1 --timeout 60s
stress-ng: info:  [10786] setting to a 1 min run per stressor
stress-ng: info:  [10786] dispatching hogs: 1 cpu
stress-ng: info:  [10786] skipped: 0
stress-ng: info:  [10786] passed: 1: cpu (1)
stress-ng: info:  [10786] failed: 0
stress-ng: info:  [10786] metrics untrustworthy: 0
stress-ng: info:  [10786] successful run completed in 1 min
rsh414@rsh414-IdeaPad-5-15ITL05:~/Daneshgah/embedded/Tamrin3_Q2$ killall procwatch
rsh414@rsh414-IdeaPad-5-15ITL05:~/Daneshgah/embedded/Tamrin3_Q2$
```

مشاهده میکنیم که برنامه متوقف میشود و پیام زیر را میدهد:

```
rsh414@rsh414-IdeaPad-5-15ITL05:~/Daneshgah/embedded/Tamrin3_Q2$ ./procwatch background src/config
rsh414@rsh414-IdeaPad-5-15ITL05:~/Daneshgah/embedded/Tamrin3_Q2$
```

```
Jul 05 15:09:48 rsh414-IdeaPad-5-15ITL05 procwatch[10301]: PID: 6856 | Name: MainThread | Activity: 2 | RSS: 157884 kB
Jul 05 15:09:48 rsh414-IdeaPad-5-15ITL05 procwatch[10301]: PID: 6967 | Name: MainThread | Activity: 1 | RSS: 86384 kB
Jul 05 15:09:48 rsh414-IdeaPad-5-15ITL05 procwatch[10301]: PID: 7076 | Name: code-fcf604774b | Activity: 1 | RSS: 20824 kB
Jul 05 15:09:48 rsh414-IdeaPad-5-15ITL05 procwatch[10301]: PID: 10301 | Name: procwatch | Activity: 1 | RSS: 4512 kB
Jul 05 15:09:48 rsh414-IdeaPad-5-15ITL05 procwatch[10301]: PID: 1 | Name: systemd | Activity: 0 | RSS: 16832 kB
Jul 05 15:09:48 rsh414-IdeaPad-5-15ITL05 procwatch[10301]: PID: 2 | Name: kthreadd | Activity: 0 | RSS: 0 kB
Jul 05 15:09:48 rsh414-IdeaPad-5-15ITL05 procwatch[10301]: PID: 14 | Name: ksoftirqd/0 | Activity: 0 | RSS: 0 kB
Jul 05 15:09:48 rsh414-IdeaPad-5-15ITL05 procwatch[10301]: PID: 15 | Name: rcu_preempt | Activity: 0 | RSS: 0 kB
Jul 05 15:09:50 rsh414-IdeaPad-5-15ITL05 procwatch[10301]:

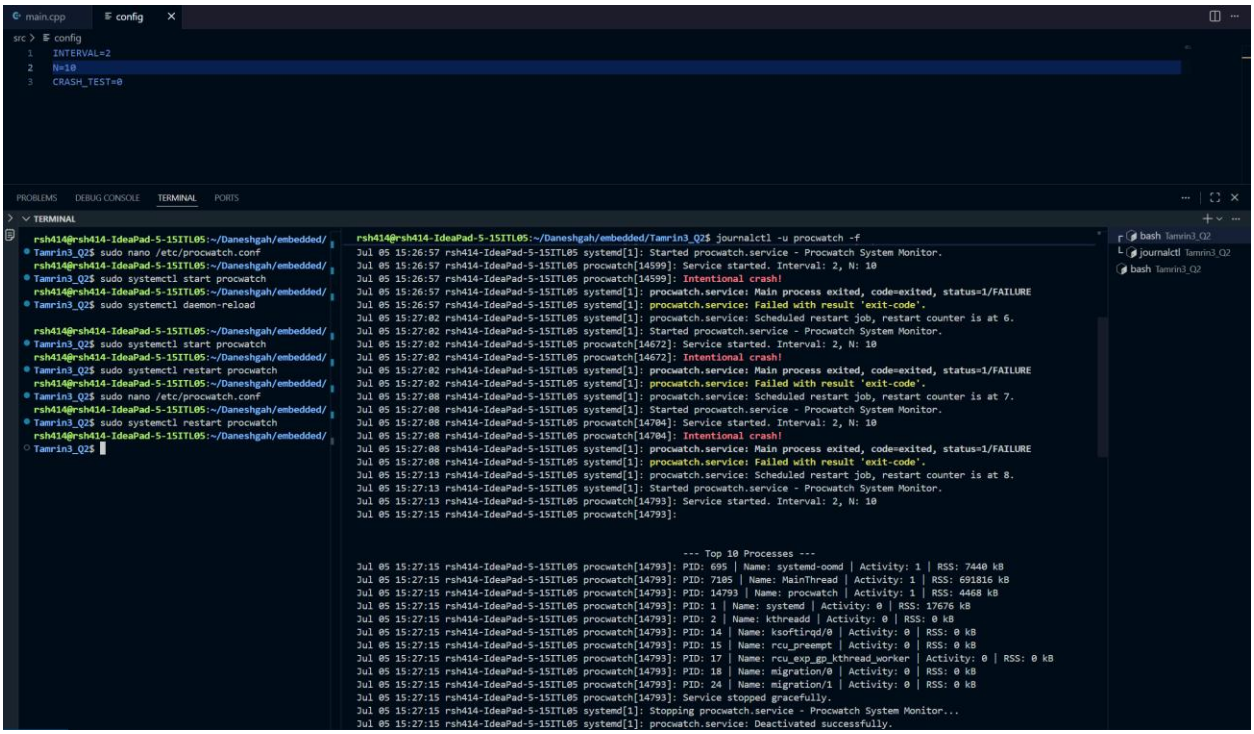
                                                    --- Top 10 Processes ---
Jul 05 15:09:50 rsh414-IdeaPad-5-15ITL05 procwatch[10301]: PID: 7105 | Name: MainThread | Activity: 4 | RSS: 664180 kB
Jul 05 15:09:50 rsh414-IdeaPad-5-15ITL05 procwatch[10301]: PID: 6856 | Name: MainThread | Activity: 2 | RSS: 157884 kB
Jul 05 15:09:50 rsh414-IdeaPad-5-15ITL05 procwatch[10301]: PID: 6967 | Name: MainThread | Activity: 2 | RSS: 86384 kB
Jul 05 15:09:50 rsh414-IdeaPad-5-15ITL05 procwatch[10301]: PID: 7057 | Name: sshd-session | Activity: 2 | RSS: 8688 kB
Jul 05 15:09:50 rsh414-IdeaPad-5-15ITL05 procwatch[10301]: PID: 7076 | Name: code-fcf604774b | Activity: 2 | RSS: 20824 kB
Jul 05 15:09:50 rsh414-IdeaPad-5-15ITL05 procwatch[10301]: PID: 1009 | Name: irq/176-iwlwifi:queue_5 | Activity: 1 | RSS: 0 kB
Jul 05 15:09:50 rsh414-IdeaPad-5-15ITL05 procwatch[10301]: PID: 10301 | Name: procwatch | Activity: 1 | RSS: 4512 kB
Jul 05 15:09:50 rsh414-IdeaPad-5-15ITL05 procwatch[10301]: PID: 1 | Name: systemd | Activity: 0 | RSS: 16832 kB
Jul 05 15:09:50 rsh414-IdeaPad-5-15ITL05 procwatch[10301]: PID: 2 | Name: kthreadd | Activity: 0 | RSS: 0 kB
Jul 05 15:09:50 rsh414-IdeaPad-5-15ITL05 procwatch[10301]: PID: 14 | Name: ksoftirqd/0 | Activity: 0 | RSS: 0 kB
Jul 05 15:09:52 rsh414-IdeaPad-5-15ITL05 procwatch[10301]:

                                                    --- Top 10 Processes ---
Jul 05 15:09:52 rsh414-IdeaPad-5-15ITL05 procwatch[10301]: PID: 6967 | Name: MainThread | Activity: 4 | RSS: 86412 kB
Jul 05 15:09:52 rsh414-IdeaPad-5-15ITL05 procwatch[10301]: PID: 10301 | Name: procwatch | Activity: 2 | RSS: 4512 kB
Jul 05 15:09:52 rsh414-IdeaPad-5-15ITL05 procwatch[10301]: PID: 6856 | Name: MainThread | Activity: 1 | RSS: 157888 kB
Jul 05 15:09:52 rsh414-IdeaPad-5-15ITL05 procwatch[10301]: PID: 7105 | Name: MainThread | Activity: 1 | RSS: 664188 kB
Jul 05 15:09:52 rsh414-IdeaPad-5-15ITL05 procwatch[10301]: PID: 1 | Name: systemd | Activity: 0 | RSS: 16832 kB
Jul 05 15:09:52 rsh414-IdeaPad-5-15ITL05 procwatch[10301]: PID: 2 | Name: kthreadd | Activity: 0 | RSS: 0 kB
Jul 05 15:09:52 rsh414-IdeaPad-5-15ITL05 procwatch[10301]: PID: 14 | Name: ksoftirqd/0 | Activity: 0 | RSS: 0 kB
Jul 05 15:09:52 rsh414-IdeaPad-5-15ITL05 procwatch[10301]: PID: 15 | Name: rcu_preempt | Activity: 0 | RSS: 0 kB
Jul 05 15:09:52 rsh414-IdeaPad-5-15ITL05 procwatch[10301]: PID: 17 | Name: rcu_exp_gp_kthread_worker | Activity: 0 | RSS: 0 kB
Jul 05 15:09:52 rsh414-IdeaPad-5-15ITL05 procwatch[10301]: PID: 18 | Name: migration/0 | Activity: 0 | RSS: 0 kB
Jul 05 15:09:54 rsh414-IdeaPad-5-15ITL05 procwatch[10301]:

                                                    --- Top 10 Processes ---
Jul 05 15:09:54 rsh414-IdeaPad-5-15ITL05 procwatch[10301]: PID: 7076 | Name: code-fcf604774b | Activity: 2 | RSS: 20824 kB
Jul 05 15:09:54 rsh414-IdeaPad-5-15ITL05 procwatch[10301]: PID: 1004 | Name: irq/171-iwlwifi:default_queue | Activity: 1 | RSS: 0 kB
Jul 05 15:09:54 rsh414-IdeaPad-5-15ITL05 procwatch[10301]: PID: 6856 | Name: MainThread | Activity: 1 | RSS: 157888 kB
Jul 05 15:09:54 rsh414-IdeaPad-5-15ITL05 procwatch[10301]: PID: 6967 | Name: MainThread | Activity: 1 | RSS: 87728 kB
Jul 05 15:09:54 rsh414-IdeaPad-5-15ITL05 procwatch[10301]: PID: 7057 | Name: sshd-session | Activity: 1 | RSS: 8688 kB
Jul 05 15:09:54 rsh414-IdeaPad-5-15ITL05 procwatch[10301]: PID: 7105 | Name: MainThread | Activity: 1 | RSS: 664188 kB
Jul 05 15:09:54 rsh414-IdeaPad-5-15ITL05 procwatch[10301]: PID: 10301 | Name: procwatch | Activity: 1 | RSS: 4512 kB
Jul 05 15:09:54 rsh414-IdeaPad-5-15ITL05 procwatch[10301]: PID: 1 | Name: systemd | Activity: 0 | RSS: 16832 kB
Jul 05 15:09:54 rsh414-IdeaPad-5-15ITL05 procwatch[10301]: PID: 2 | Name: kthreadd | Activity: 0 | RSS: 0 kB
Jul 05 15:09:54 rsh414-IdeaPad-5-15ITL05 procwatch[10301]: PID: 14 | Name: ksoftirqd/0 | Activity: 0 | RSS: 0 kB
Jul 05 15:09:56 rsh414-IdeaPad-5-15ITL05 procwatch[10301]: Service stopped gracefully.
```

اکنون در ***config***، ***CRASH_TEST*** را برابر 1 قرار میدهیم:

مشاهده میکنیم که سیستم هر 5 ثانیه خود را ری استارت میکنم ولی دوباره با پارامتر ***CRASH_TEST = 1*** روبرو میشود و فیل میشود



سپس با تغییر ***CRASH_TEST***=0 مشاهده میکنیم که سیستم دوباره شروع به ذخیره لاگ بصورت عادی میکند.